AI Newsletter

Complete Project Documentation

Architecture | Code Reference | Study Guide

Full-stack FastAPI + MongoDB + OpenAI LLM Newsletter Platform

Vanilla HTML/CSS/JS Frontend | 3 Themes | Google OAuth

Table of Contents

1. Project Overview

AI Newsletter (AI Pulse) is a full-stack web application that automatically curates AI/ML industry content into professional themed weekly digests. It scrapes real news from 7 RSS feeds and 2 job board APIs, uses OpenAI GPT with structured outputs to curate and summarize them into 5 thematic sections, stores complete editions in MongoDB, and serves them via a FastAPI backend with Jinja2 server-side rendered templates and vanilla HTML/CSS/JS frontend.

Key capabilities: automated newsletter generation via LLM, real AI news scraping, curated job board with experience tiers, 3-theme support (light/dark/warm), full-text archive search, Google OAuth login, send-to-self email sharing via Gmail API, and rate limiting.

Core Principles (AGENTS.md)

- File Size Discipline — No file exceeds 500 lines
- Modu

2. Architecture & Layer Diagram

The project follows a strict 4-Layer Modular Architecture combined with MVC principles. Each layer has a single responsibility and only depends on the layers below it.

Layer	Folder	Responsibility
Config	backend/config/	Environment variables, Jinja2 instance, shared settings
Database	backend/database/	MongoDB async connection, indexing, lifecycle management
Models	backend/models/	Pydantic schemas for validation and serialization
Services	backend/services/	Business logic: scraping, LLM calls, CRUD, email, rate limiting
Routers	backend/routers/	HTTP endpoints (REST APIs + HTML page rendering)
Frontend	frontend/	Templates, CSS, JavaScript — all vanilla, no frameworks

Layer Diagram

```
FRONTEND (Presentation Layer)
  HTML Templates + Vanilla JS + CSS (3 themes)
          |
  --------+--------
  |                |
  API Routers      Page Routers
  (REST JSON)      (Jinja2 HTML)
  --------+--------
          |
  SERVICES LAYER (Business Logic)
  content_generator | email_service | llm_client
  news_scraper | jobs_scraper | jobs_service
  newsletter_service | rate_limiter
          |
  MODELS LAYER (Data Validation)
  newsletter.py | job.py | share.py
          |
  CONFIG / DATABASE LAYER
  settings.py | connection.py | templates.py
```

3. Data Flow Diagrams

3.1 Newsletter Generation Flow (Admin Only)

```
POST /api/v1/newsletter/generate  [X-API-Key header]
  |--> Acquire asyncio.Lock (prevent concurrent generation)
  |--> Scrape real AI news from 7 RSS feeds
  |     (TechCrunch, The Verge, VentureBeat, Ars Technica,
  |      WIRED, MIT Tech Review, Google AI Blog)
  |--> For each of 5 sections (sequential):
  |     |-- Format articles as LLM context
  |     |-- Call OpenAI GPT with structured output
  |     +-- Validate Pydantic response --> ContentItem[]
  |--> Jobs Board special handling:
  |     |-- Scrape RemoteOK + Arbeitnow APIs
  |     |-- Filter by AI/ML keywords
  |     +-- LLM curates --> JobListing[]
  |--> Generate edition headline + executive summary via LLM
  +--> Save NewsletterEdition to MongoDB (status=published)
```

3.2 Serving Flow (Public)

```
GET / (Homepage)
  +--> Fetch latest published edition from MongoDB
       +--> Render with index.html + 5 section partials

GET /archive (Archive)
  +--> Fetch paginated editions --> Render archive.html

GET /edition/{id}
  +--> Fetch by ObjectId --> Render edition.html
```

3.3 Email Sharing Flow (Authenticated)

```
GET  /auth/login       --> Redirect to Google OAuth consent
GET  /auth/callback    --> Exchange code --> Set signed session cookie
POST /api/v1/share/send-to-self
  +--> Extract user from cookie --> Rate limit check
       +--> Fetch edition --> Build HTML email --> Gmail API send
```

3.4 Theme Persistence

```
Page Load --> Inline <head> script reads localStorage
          --> Applies data-theme attr BEFORE render (no flash)
Click toggle --> Cycle: light --> dark --> warm --> light
             --> Update localStorage + meta theme-color
```

4. Backend Files — Detailed Breakdown

4.1 Configuration Layer

File	Purpose	Key Exports	Topics to Study
backend/config/settings.py	Loads all env vars into typed Settings object using Pydantic BaseSettings	Settings class — fields for MongoDB URI, OpenAI key, OAuth, session, rate limits	Pydantic Settings, BaseSettings, dotenv, 12-Factor App
backend/config/templates.py	Shared Jinja2Templates singleton to avoid circular imports	templates — global Jinja2Templates instance	Jinja2 Templating, Singleton pattern, Python pathlib

4.2 Database Layer

File	Purpose	Key Functions	Topics to Study
backend/database/connection.py	MongoDB async connection with lifespan mgmt; creates indexes on startup	db_lifespan(app) get_database() get_collection(name) Indexes: edition_number (unique), [status, created_at]	pymongo AsyncMongoClient, FastAPI Lifespan, MongoDB Indexes, Context Managers, certifi SSL

4.3 Models Layer (Pydantic Schemas)

File	Purpose	Key Classes	Topics to Study
backend/models/newsletter.py (~87 lines)	Core data models for editions, sections, content items, archive responses	SectionType (enum) EditionStatus (enum) ContentItem NewsletterSection NewsletterEdition ArchiveEditionSummary ArchiveResponse PyObjectId	Pydantic v2 Models, Enums, Field aliases (_id), BeforeValidator, MongoDB ObjectId, Annotated types
backend/models/job.py (~42 lines)	Job listing schema with location and experience enums	LocationType ExperienceTier JobListing (with min/max length validation)	Pydantic Field Constraints, str Enums, Data Validation patterns
backend/models/share.py (~24 lines)	Request/response DTOs for email sharing endpoint	SendToSelfRequest ShareEmailResponse	Pydantic DTOs, API Request/Response design

4.4 Services Layer (Business Logic)

File	Purpose	Key Functions	Ext. APIs	Topics to Study
content_generator.py (~312 lines)	Orchestrates section-by-section LLM newsletter generation	generate_full_edition() generate_section() _generate_jobs_section() generate_headline()	OpenAI GPT	LLM Orchestration, Structured Outputs, Prompt Engineering
llm_client.py (~105 lines)	OpenAI API client with structured JSON output + Pydantic validation	generate_structured() _build_schema_instruction() _extract_json()	OpenAI API	OpenAI json response_format, JSON Schema, httpx async
news_scraper.py (~138 lines)	Scrapes 7 RSS feeds for AI news articles	scrape_ai_news() _fetch_feed() format_articles_for_prompt()	7 RSS feeds	feedparser, HTML sanitization, Multi-source aggregation
jobs_scraper.py (~178 lines)	Scrapes RemoteOK + Arbeitnow for AI/ML jobs	scrape_ai_jobs() _fetch_remoteok_jobs() _fetch_arbeitnow_jobs()	RemoteOK Arbeitnow	Web Scraping APIs, Keyword filtering, Deduplication
jobs_service.py (~58 lines)	Bridges scraping to LLM curation for job listings	generate_job_listings() _fallback_generate()	OpenAI	Service composition, Fallback patterns
newsletter_service.py (~225 lines)	MongoDB CRUD for newsletter editions	create_edition() get_latest() get_by_id() list_paginated() search_editions() get_available_years()	MongoDB	Repository Pattern, Aggregation Pipelines, Pagination
email_service.py (~268 lines)	Builds HTML+text email, sends via Gmail API	send_newsletter_email() _build_html() _build_plain_text()	Gmail API	Gmail API (OAuth), MIME, HTML email, Base64
rate_limiter.py (~40 lines)	In-memory sliding-window rate limiter per user	check_rate_limit(key) reset()	None	Sliding Window algorithm, Time-based quota

4.5 Routers Layer (HTTP Endpoints)

File	Purpose	Key Endpoints	Auth	Topics to Study
routers/newsletter.py (~106 lines)	Newsletter generation + retrieval API	POST .../generate GET .../latest GET .../archive GET .../search GET .../{id}	API Key (X-API-Key)	FastAPI APIKeyHeader, asyncio.Lock, secrets.compare_digest
routers/auth.py (~188 lines)	Google OAuth 2.0 login/logout/session	GET /auth/login GET /auth/callback POST /auth/logout GET /auth/me	Google OAuth 2.0	OAuth 2.0 AuthCode Flow, CSRF state, Signed cookies (itsdangerous)
routers/share.py (~66 lines)	Send-to-self email endpoint	POST /api/v1/share/send-to-self	Session cookie	Rate limiting, Authenticated endpoints, Error patterns (401/429/404)
routers/pages.py (~30 lines)	Server-side rendered HTML pages	GET / (homepage) GET /archive GET /edition/{id}	None (public)	Jinja2 TemplateResponse, Server-side rendering
routers/health.py (~26 lines)	Health check endpoint	GET /api/v1/health	None	Health check pattern, DB ping, Monitoring

4.6 Application Entry Point

File	Purpose	Key Components	Topics to Study
backend/main.py (~37 lines)	FastAPI app factory; mounts static files, includes all routers, sets database lifespan	app = FastAPI(lifespan=db_lifespan) app.mount('/static') app.include_router(...) x5	FastAPI Application Factory, Static file mounting, Router composition, Lifespan events

5. Frontend Files — Detailed Breakdown

5.1 Templates (Jinja2)

File	Purpose	Key Features	Topics to Study
base.html	Base layout — HTML head, CSS/JS imports, blocks	SEO meta, OpenGraph, inline theme script, 6 CSS + 2 JS	Jinja2 Template Inheritance, FOUC prevention
index.html	Homepage — latest edition with all sections	Extends base, includes 5 section partials conditionally	Jinja2 includes, Conditional rendering
archive.html	Archive — search, filter, paginated editions	Custom dropdowns, search input, pagination, archive.js	AJAX-driven filtering, Pagination UI
edition.html	Single edition view	Same as index + Back to Archive nav	Deep linking, Template reuse
empty.html	Empty state fallback (no editions)	Welcome message + call to action	Fallback state pattern

5.2 Template Partials (frontend/templates/partials/)

File	Purpose	Topics to Study
header.html	Sticky header: logo, nav, theme toggle, auth UI (login/profile dropdown)	Reusable components, Conditional UI
footer.html	Copyright footer with dynamic year	Template partials
section_trending.html	Trending AI Topics — collapsible, content cards, share button	Collapse/expand UI, Jinja2 loops
section_developments.html	Top Developments — same pattern as trending	Reusable section layout
section_tools.html	Corporate AI Tools section	Section component pattern
section_future.html	Future Requirements & Trends section	Section component pattern
section_jobs.html	Jobs Board — two tiers (1-2yr / 2-4yr), job cards + Apply link	Jinja2 selectattr, Tiered display
share_button.html	Reusable clipboard share button with Copied! feedback	Micro-component reuse

5.3 JavaScript (Vanilla — No Framework)

File	Purpose	Key Functions	Topics to Study
theme.js (~72 ln)	Theme switcher: Light/Dark/Warm + localStorage	getTheme(), applyTheme(), cycleTheme()	CSS Custom Prop theming, localStorage, data-* attrs
auth.js (~240 ln)	Auth state, login/logout UI, send-to-self, toasts	checkAuthState(), bindSendToSelf(), showToast()	Fetch API, Event delegation, Toast pattern
sections.js (~63 ln)	Section expand/collapse with persistent state	initSections(), getExpandedState()	CSS class toggling, State persistence
archive.js (~169 ln)	Archive search + custom dropdowns + highlighting	initCustomSelects(), doSearch(), highlightText()	Debounced search, AJAX, Regex highlighting
share.js (~44 ln)	Copy section deep-link to clipboard with fallback	navigator.clipboard.writeText()	Clipboard API, Feature detection
smooth-scroll.js (~30 ln)	Smooth scroll to anchors on load and click	scrollIntoView({behavior: 'smooth'})	Smooth scroll API, History API

5.4 CSS (Design System)

File	Purpose	Key Concepts	Topics to Study
reset.css (~33 ln)	CSS reset/normalize	* { box-sizing: border-box }, font inherit	CSS Reset, Box model
typography.css (~67 ln)	Fonts, heading scale, theme overrides	Google Fonts, h1-h6, [data-theme] selectors	Web fonts, Typographic scale
layout.css (~115 ln)	Core layout: design tokens, header, grid	--color-bg, --spacing-md, sticky header	CSS Custom Properties, Sticky
sections.css (~150+ ln)	Section cards, collapse/expand, colors	max-height transition, colored pills, hover	CSS Transitions, Pseudo-elements
auth.css (~156+ ln)	Auth UI: login, avatar, dropdown, toast	Slide animation, @keyframes, ARIA	CSS Animations, Accessibility
responsive.css (~97 ln)	Media queries: mobile/tablet/desktop/print	@media (max-width: 767px), @media print	Mobile-first, Print styles

6. API Endpoints Summary

Method	Endpoint	Auth	Purpose
POST	/api/v1/newsletter/generate	API Key	Generate new newsletter edition via LLM
GET	/api/v1/newsletter/latest	None	Get latest published edition (JSON)
GET	/api/v1/newsletter/archive	None	Paginated published editions (JSON)
GET	/api/v1/newsletter/search	None	Search editions by text/date (JSON)
GET	/api/v1/newsletter/months	None	Get available years (JSON)
GET	/api/v1/newsletter/{id}	None	Get specific edition by ID (JSON)
GET	/api/v1/health	None	Health check with database ping
GET	/auth/login	None	Redirect to Google OAuth consent
GET	/auth/callback	None	OAuth callback — exchange code for token
POST	/auth/logout	Session	Clear session cookie
GET	/auth/me	Session	Get current user info
POST	/api/v1/share/send-to-self	Session	Email newsletter to logged-in user
GET	/	None	Homepage (HTML)
GET	/archive	None	Archive page (HTML)
GET	/edition/{id}	None	Single edition page (HTML)

7. Security Model

Security Layer	Implementation	File
API Key Auth	APIKeyHeader + secrets.compare_digest() (constant-time)	routers/newsletter.py
OAuth 2.0	Google Auth Code Flow + CSRF state parameter	routers/auth.py
Session Cookies	Signed with itsdangerous TimestampSigner, httponly, samesite=lax	routers/auth.py
Rate Limiting	Sliding-window per user email (3 req / 600 sec)	services/rate_limiter.py
Concurrency Guard	asyncio.Lock prevents simultaneous generation	routers/newsletter.py
Secrets Mgmt	All credentials via .env file, never hardcoded	config/settings.py
MongoDB Indexes	Query efficiency and edition_number uniqueness	database/connection.py

8. Design Patterns Used

Pattern	Where Used	Purpose
4-Layer Architecture	Entire backend	Strict separation: config → models → services → routers
Repository Pattern	newsletter_service.py	Abstracts MongoDB behind clean functions
Facade Pattern	content_generator.py	Orchestrates scrapers + LLM + validation
Singleton	settings.py, templates.py, connection.py	Single shared instances across the app
Dependency Injection	FastAPI lifespan, Security headers	Framework-managed dependency resolution
Strategy Pattern	news_scraper, jobs_scraper	Multiple data sources as interchangeable inputs
Sliding Window	rate_limiter.py	Time-based quota enforcement per user
OAuth 2.0 AuthCode Flow	auth.py	Secure token exchange with signed cookies
Template Inheritance	Jinja2 base.html → children	DRY HTML structure with named blocks
Event Delegation	All JS files	Single parent handler for dynamic children
State Persistence	theme.js, sections.js, auth.js	localStorage for UI preferences
Graceful Degradation	content_generator, scrapers	Fallback content when APIs fail

9. Technology Stack

Category	Technology	Version	Why
Backend Framework	FastAPI	0.128	Async, auto OpenAPI docs, dependency injection
ASGI Server	Uvicorn	0.39	High-performance async Python server
Database	MongoDB (pymongo async)	4.16	Document structure fits nested newsletter schema
LLM	OpenAI GPT	—	Structured outputs: guaranteed valid JSON
HTTP Client	httpx	0.28	Async HTTP for scraping + API calls
RSS Parsing	feedparser	6.0	Robust RSS/Atom feed parsing
Templates	Jinja2	3.1	Server-side HTML rendering
Config	pydantic-settings	2.11	Type-safe environment configuration
Sessions	itsdangerous	2.2	Signed cookie sessions without DB
SSL	certifi	2026	SSL certificates for MongoDB Atlas
Frontend	Vanilla HTML/CSS/JS	—	No frameworks — lightweight and fast

10. Environment Configuration (.env)

Variable	Purpose	Example
MONGODB_URI	MongoDB connection string	mongodb+srv://user:pass@cluster
MONGODB_DB_NAME	Database name	ai-newsletter
OPENAI_API_KEY	OpenAI API key for LLM	sk-...
OPENAI_MODEL	GPT model name	gpt-4o
ADMIN_API_KEY	API key for /generate endpoint	random 32-char token
APP_ENV	Environment mode	development / production
APP_PORT	Server port	8000
GOOGLE_CLIENT_ID	Google OAuth client ID	xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET	Google OAuth client secret	GOCSPX-...
GOOGLE_REDIRECT_URI	OAuth callback URL	http://localhost:8000/auth/callback
SESSION_SECRET_KEY	Cookie signing key	random secret string
SESSION_MAX_AGE	Session TTL in seconds	86400 (24 hours)
RATE_LIMIT_MAX_REQUESTS	Max emails per window	3
RATE_LIMIT_WINDOW_SECONDS	Rate limit window	600 (10 minutes)

11. How to Run

Step 1: Install Dependencies

Step 2: Configure Environment

Step 3: Start the Server

Step 4: Generate First Newsletter

Step 5: View

Open http://localhost:8000 in your browser.

12. Study Topics Reference

To fully understand this codebase, study these topics organized by priority level:

Priority	Topic	Relevant Files
MUST-KNOW	FastAPI (routing, DI, lifespan)	main.py, all routers
MUST-KNOW	Pydantic v2 (models, validators, BaseSettings)	all models, settings.py
MUST-KNOW	MongoDB CRUD + Aggregation (pymongo async)	connection.py, newsletter_service.py
MUST-KNOW	Python async/await + asyncio (Lock, gather)	All services and routers
MUST-KNOW	Jinja2 template inheritance + includes	All templates
IMPORTANT	OAuth 2.0 Authorization Code Flow	auth.py
IMPORTANT	OpenAI API (structured outputs, response_format)	llm_client.py, content_generator.py
IMPORTANT	RSS Feed Parsing with feedparser	news_scraper.py
IMPORTANT	REST API Design (CRUD, pagination, search)	newsletter.py router + service
IMPORTANT	Async HTTP client (httpx)	llm_client.py, scrapers, email_service.py
GOOD-TO-KNOW	CSS Custom Properties + Multi-theme support	All CSS files
GOOD-TO-KNOW	Vanilla JS: DOM, Fetch API, Event delegation	All JS files
GOOD-TO-KNOW	Sliding Window Rate Limiting algorithm	rate_limiter.py
GOOD-TO-KNOW	Gmail API (OAuth token-based sending)	email_service.py
GOOD-TO-KNOW	Cookie sessions (itsdangerous signing)	auth.py
GOOD-TO-KNOW	Web Scraping (API-based, keyword filtering)	jobs_scraper.py
GOOD-TO-KNOW	Responsive CSS + Mobile-first design	responsive.css
GOOD-TO-KNOW	MIME email (HTML + plaintext multipart)	email_service.py